

Assignment 10

Task 1 If Statements and Conditions

Determine the sequence of MJVM instructions generated by the attributed grammar presented on lecture slides 70–74 for the following if statement. For each generated instruction, give the production and semantic action of the attributed grammar that emits the instruction. For jump instructions, also explain when the target address is written to the code buffer. x and y are global variables at addresses 0 and 1, respectively.

```
if (x < 0 || x > 9 && x < 100) y = 1;
else y = 0;
```

1. Theoretical Background

The goal of this task is to translate a compound Boolean condition in an if statement into MJVM bytecode. The expression is evaluated using short-circuit semantics based on slides 70–74 of the Code Generation lecture. Conditions are translated using `Operand.Cond` objects that carry fixup labels for true and false outcomes. Boolean operators `||` and `&&` are handled using conditional jumps (`tJump` and `fJump`).

Conditions



if (a > b) ...

Condition

code pattern

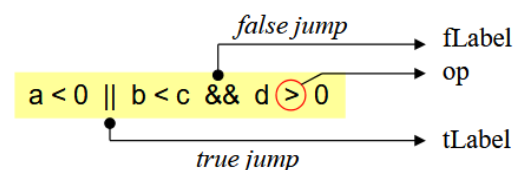
load a
load b
jle ...

- Problem: there is no compare instruction in the μ JVM (the comparison only happens in the jump instruction)
- *Condition* thus cannot do the comparison
=> returns a compare operator instead

Cond operand

Condition returns a new operand kind with the following fields:

- compare operator: eq, ne, lt, le, gt, ge
- labels (fixup lists) for jumps out of the *Condition*
 - tLabel: for *true jumps*
 - fLabel: for *false jumps*



necessary for conditions that are built with `&&` and `||`

True jumps and false jumps

true jump jumps if the condition is true $a > b \Rightarrow \text{jgt ...}$
false jump jumps if the condition is false $a > b \Rightarrow \text{jle ...}$

64

Figure 1: Slide 64

2. MicroJava Virtual Machine (MJVM) bytecode sequence

The conditional statement `if (x < 0 || x > 9 && x < 100) y = x; else y = 0;` involving global variables `x` at data address 0 and `y` at data address 1 translates into the following MicroJava Virtual Machine (MJVM) bytecode sequence:

Address	Instruction	Bytes	Explanation	Estack
0	getstatic 0	3	Load global variable x	
3	const0	1	Load constant 0	
4	jlt	3	True Jump (): if x < 0, bypass remaining conditions to If-body	
7	getstatic 0	3	Load global variable x	
10	const 9	5	Load constant 9 (1 byte opcode + 4 bytes value = 5 bytes)	
15	jle	3	False Jump (&&): if x <= 9, short-circuit to Else-body	
18	getstatic 0	3	Load global variable x	
21	const 100	5	Load constant 100 (1 byte opcode + 4 bytes value = 5 bytes)	
26	jge	3	False Jump (end of condition): if x >= 100, jump to Else-body	
29	getstatic 0	3	If-body: Load global variable x	
32	putstatic 1	3	Assign to global variable y	
35	jmp	3	Unconditional jump over the Else-body	
38	const0	1	Else-body: Load constant 0	
39	putstatic 1	3	Assign to global variable y	
42				Next instruction after the conditional structure

	Address	Instruction	Bytes	Explanation	Estack
x<0	0	getstatic 0	3	Load global variable x	
	3	const0	1	Load constant 0	
x>9	4	jlt 25 (=29)	3	True Jump (): if x < 0, bypass remaining conditions to If-body	
	7	getstatic 0	3	Load global variable x	
x<100	10	const 9	5	Load constant 9 (1 byte opcode + 4 bytes value = 5 bytes)	
	15	jle 23 (=38)	3	False Jump (&&): if x <= 9, short-circuit to Else-body	
IF	18	getstatic 0	3	Load global variable x	
	21	const 100	5	Load constant 100 (1 byte opcode + 4 bytes value = 5 bytes)	
IF	26	jge 12 (=38)	3	False Jump (end of condition): if x >= 100, jump to Else-body	
	29	getstatic 0	3	If-body: Load global variable x	
ELSE	32	putstatic 1	3	Assign to global variable y	
	35	jmp 7 (=42)	3	Unconditional jump over the Else-body	
ELSE	38	const0	1	Else-body: Load constant 0	
	39	putstatic 1	3	Assign to global variable y	
	42				Next instruction after the conditional structure

3. Step-by-Step Translation and Structural Attribution

1. Condition Component: `x < 0`

- **Production:** `CondFactor = Expr Relop Expr`.
- **Semantic Action:** Emits `getstatic 0` and `const0`. The relational operator `lt (<)` is passed up.
- **Short-Circuit Logic (||):** Evaluated via `Condition = CondTerm {"||\vdots"}`. Since a logical OR follows, a *True Jump* (`Code.tJump`) is generated using the direct operator `jlt`.
- **Target Resolution:** The `x.fLabel.here()` action immediately follows the `||` operator, establishing that if `x < 0` evaluates to false, control flow drops directly into the next instruction block (PC 7).

2. Condition Component: `x > 9`

- **Production:** `CondFactor = Expr Relop Expr`.
- **Semantic Action:** Emits `getstatic 0` and `const 9`. The relational operator `gt (>)` is passed up.
- **Short-Circuit Logic (&&):** Evaluated via `CondTerm = CondFactor {"&&\vdots"}`. Since a logical AND follows, a *False Jump* (`Code.fJump`) is generated. It uses the inverse operator `jle (<=)`. If false, it routes control flow directly to the `fLabel` list (Else-body).

3. Condition Component: $x < 100$

- **Production:** `CondFactor = Expr Relop Expr`.
- **Semantic Action:** Emits `getstatic 0` and `const 100`. The operator `lt (<)` is passed up.
- **Closing Parenthesis `)`:** Evaluated in `IfStatement = "if" "(" Condition ")" Statement`. A final *False Jump* (`Code.fJump`) is emitted for the terminal condition member using the inverse operator `jge ( $\geq$ )`.
- **Target Resolution:** Right after this jump, `x.tLabel.here()` is executed. This anchors the target location for the early logical OR `jlt` instruction (PC 4) to the beginning of the `If-body` (PC 29).

4. Statement execution: `y = x;`

- **Production:** `Statement = Designator "=" Expr ";"`.
- **Semantic Action:** Emits `getstatic 0` followed by `putstatic 1`.
- **Structural Transition:** Prior to entering the `else` branch, the parser instantiates an unconditional forward label (`end`) and emits `Code.jump(end) (jmp)`. Right after, `x.fLabel.here()` is called, pinpointing the starting offset of the `Else-branch` (PC 32) and resolving all pending false jump points.

5. Statement execution: `y = 0;`

- **Production:** `Statement = Designator "=" Expr ";"`.
- **Semantic Action:** Emits `const0` followed by `putstatic 1`.
- **Structural Closure:** At the end of the `IfStatement` parsing routine, `end.here()` is invoked. This binds the terminal offset (PC 39) to the forward jump emitted at PC 35.

How the Compiler Sneaks the Fixup List Into the Bytecode

Instead of managing a massive separate array of integers to keep track of every unpatched jump, the `Label` object in the compiler only tracks a single integer: the address of the most recent jump instruction that needs fixing. The older unpatched jump instructions literally hold the address of the next instruction waiting in line, forming a chain directly inside the binary code array! Here is how that "unknowingness" gets chained together and patched up:

Imagine we are compiling our condition, and multiple instructions need to jump to the `else-body` (our `fLabel`)

- At PC 15: The compiler emits `jle`. It doesn't know where the `else-body` is yet. The `fLabel` currently stores `-1` (meaning the end of the chain). The compiler writes `-1` as the operand for `jle`, and updates `fLabel` to point to 15.
- At PC 26: The compiler emits `jge`. It still doesn't know where the `else-body` is. It looks at `fLabel`, sees it currently points to 15, so it writes 15 as the operand for `jge`. Then it updates `fLabel` to point to 26.

PC 15: `jle -1` (Points to the end of the chain)

PC 26: `jge 15` (Points backward to the previous unresolved jump!)

The `Label` object simply stores 26

The Patch-Up Phase (`here()`)

When the parser finally hits PC 38 (the actual start of the `else-body`), you call `x.fLabel.here()`. This triggers a loop inside the compiler called backpatching.

- The compiler looks at `fLabel` and sees the number 26.
- It goes to PC 26 in the bytecode buffer, reads its current placeholder value (15), and overwrites the operand at PC 26 with the true address: 38.
- It moves to the next link in the chain, PC 15. It reads its placeholder value (`-1`). It overwrites the operand at PC 15 with the true address: 38.
- It sees `-1` and knows the chain is completely fixed up!

Task 2 Switch statements – ATG

Write a production `SwitchStatement` of an attributed grammar for translating a switch statement to MicroJava byte code. An example of a switch statement is given below (variable declarations are not shown). The switch expression should be restricted to the integer data type, and case labels should be numbers (constants). Cases need to contain break statements unless control flow should "fall through" to the next case - this is the same semantics as for well-known programming languages like Java or C. Note that the MicroJava compiler already supports break statements - no modification needed (see lecture slide 72).

```
switch (x % 5) {  
  case 0: y = x / 5; n0++; break;  
  case 1: y = (x-1) / 5; n1++; break;  
  case 2: n2++; // fall through  
  default: y = x;  
}
```

Solution

Following the strict one-pass compilation architecture of the MicroJava compiler, the translation of a switch statement is split into two sequential phases: the **conditional evaluation chain** and the **statement execution bodies**.

- **Type Verification:** The compiler ensures that the type of the control expression evaluates to an integer (`Tab.intType`), and that each accompanying case label resolves to a numeric constant.
- **Value Duplication:** Evaluating the switch expression places its runtime value onto the topmost position of the Expression Stack (`EStack`). Because a conditional jump instruction (`jeq`) pops the top two elements from the stack to perform a comparison, the runtime value must be duplicated via the `dup` opcode before executing each individual case check.
- **Target Backpatching via Forward Labels:** Since case bodies appear lower in the source file, their physical bytecode addresses are unknown during the initial construction of the evaluation chain. The compiler instantiates a distinct forward `Label` for every case branch, routing successful matches through a conditional `Code.tJump (jeq)`.
- **Fall-Through and Break Resolution:** If a conditional match fails, control flow naturally drops into the subsequent case comparison. Once a match is triggered, execution enters its respective body block. If a body omits a `break` statement, execution sequentially overflows into the subsequent bytecode instructions (*fall-through* semantics). `break` statements leverage the pre-existing loop-handling tracking system by pushing the terminal `endSwitch` label onto the active breaks stack.

```
Statement = Designator ( Assignop Expr | ActPars | "++" | "--" ) ";"  
          | "if" "(" Condition ")" Statement [ "else" Statement ]  
          | "while" "(" Condition ")" Statement  
          | "switch" "(" Expr ")" [ { "case" Expr ":" {Statement} }  
                                   [ "default" ":" {Statement} ] ]  
          | "break" ";"  
          | "return" [ Expr ] ";"  
          | "read" "(" Designator ")" ";"  
          | "print" "(" Expr [ ", number ] ")" ";"  
          | Block  
          | ";"
```

Figure 2: [https://ssw.jku.at/Teaching/Lectures/UB/UE/2014/MicroJava-Grammatik\(Folie\).pdf](https://ssw.jku.at/Teaching/Lectures/UB/UE/2014/MicroJava-Grammatik(Folie).pdf)

2. Attributed Grammar (ATG) Production Specification

Below is the complete production rule SwitchStatement designed for integration into the MicroJava compiler specification.

```
1 SwitchStatement
2 (.
3     Operand expr;
4     int val;
5     Label defaultLab = new Label();
6     Label endSwitch = new Label();
7     ArrayList<Integer> caseValues = new ArrayList<>();
8     ArrayList<Label> caseLabels = new ArrayList<>();
9     boolean hasDefault = false;
10
11     Label oldBreak = breakLab;
12     breakLab = endSwitch;
13 .)
14 =
15 "switch" "(" Expr<out expr>
16 (.
17     if (expr.type != Tab.intType) {
18         error("switch expression must be of type int");
19     }
20     Code.load(expr);
21 .)
22 ")" "{"
23 {
24     "case" number
25     (.
26         val = token.val;
27         if (caseValues.contains(val)) {
28             error("duplicate case label");
29         }
30         Label caseBody = new Label();
31         caseValues.add(val);
32         caseLabels.add(caseBody);
33
34         Code.put(Code.dup);
35         Code.load(new Operand(val));
36         Code.tJump(Code.eq, caseBody);
37     .)
38     ":"
39 }
40 (. Code.jump(defaultLab); .)
41 {
42     "case" number
43     (.
44         val = token.val;
45         int idx = caseValues.indexOf(val);
46         if (idx >= 0) {
47             caseLabels.get(idx).here();
48         }
49     .)
50     ":" { Statement }
51
52 |
53
54 "default" ":"
55 (.
56     hasDefault = true;
57     defaultLab.here();
58 .)
59 { Statement }
60 }
```

```

61 "}"
62 (.
63   if (!hasDefault) {
64     defaultLab.here();
65   }
66   Code.put(Code.pop);
67   endSwitch.here();
68   breakLab = oldBreak;
69 .)
70 .

```

3. Trace Verification of the Generated Bytecode Scheme

Using the sample specification `switch (x % 5)`, the compiled instruction layout strictly matches the following internal execution pipeline:

1. **Evaluation Chain Setup:** The modulo expression is evaluated, placing its result on the expression stack.
2. **Sequential Verification Cascade:**
 - `dup` → `const 0` → `jeq CASE_0_BODY` (Consumes duplicated value and constant).
 - `dup` → `const 1` → `jeq CASE_1_BODY`.
 - `dup` → `const 2` → `jeq CASE_2_BODY`.
 - `jmp DEFAULT_BODY` (Fallback execution branch).
3. **Isolated Execution Branches:**
 - `CASE_0_BODY`: Executes `y = x / 5; n0++;` → hits `break` → emits `jmp END_SWITCH`.
 - `CASE_1_BODY`: Executes `y = (x - 1) / 5; n1++;` → hits `break` → emits `jmp END_SWITCH`.
 - `CASE_2_BODY`: Executes `n2++`. Since no `break` statement is specified, control flow drops directly into the subsequent memory byte address (*fall-through* behavior into `DEFAULT_BODY`).
 - `DEFAULT_BODY`: Executes `y = x;`.
4. **Final Stack Clearance:** At the targeted instruction offset (`END_SWITCH`), the terminal `Code.put (Code.pop)` is evaluated, clearing the original evaluation value off the stack and returning the environment to an empty state.

```

--- EVALUATION CASCADE ---
dup                // Copy the result of (x % 5)
const 0            // Load case value 0
jeq CASE_0_BODY    -----// If equal, jump directly to body
|
dup                | // (Skipped if jumped) Copy again
const 1            |
jeq CASE_1_BODY    -----+-----// If equal, jump directly to body
|                    |
jmp DEFAULT_BODY   -----+-----+-----// Fallback if no case matches
|                    |
--- EXECUTION BODIES --- |
CASE_0: y = x / 5; n0++; |
      jmp END_SWITCH <-----+ | // break statement!
|                               |
CASE_1: y = (x - 1) / 5; n1++; |
      jmp END_SWITCH <-----+ | // break statement!
|                               |
CASE_2: n2++; // NO BREAK! Fall-through! |
DEFAULT: y = x; <-----+
|
END_SWITCH:
pop                // Clean up the original expression value!

```

Task 4 Method declaration and calls

Compile the following source program using the provided (unmodified) MJ compiler and decode the generated object file using MJ.Decode. Locate and explain the MJVM instructions enter, exit, return, and call (including operand values) in the decoded output. Then execute the object file using MJ.Run with -debug option, and explain what happens on method and expression stack when method sum2 is called and when it returns to the caller.

```
program MethodCall
{
    int sum2(int min, int max)
    int i, s;
    {
        s = 0;
        i = min;
        while (i <= max) {
            s = s + i*i;
            i++;
        }
        return s;
    }
    void main()
    {
        print(sum2(2,4));
    }
}
```

Address	Instruction	Length	Explanation	EStack
29	enter 0 0	3	Allocates a frame for main (0 params, 0 locals).	[]
32	const2	1	Pushes the constant value 2 onto the stack.	[2]
33	const4	1	Pushes the constant value 4 onto the stack.	[2, 4]
34	call -34	3	Pushes return address 37 to MStack, jumps to 37 - 34 = 3.	[2, 4]

Figure 3: main method

0	enter 2 4	3	Pops arguments into locals: min=2 (slot 0), max=4 (slot 1).	[]
3	const0	1	Pushes 0 onto the stack.	[0]
4	store3	1	Pops 0 and stores it in variable s (slot 3).	[]
5	load0	1	Loads the value of min (slot 0) onto the stack.	[2]
6	store2	1	Pops 2 and stores it in loop counter i (slot 2).	[]
7	load2	1	Loads i (value: 2) onto the stack.	[2]
8	load1	1	Loads max (value: 4) onto the stack.	[2, 4]
9	jgt 15	3	Compares if i > max (2 > 4). False, so it falls through.	[]
12	load3	1	Loads s (value: 0) onto the stack.	[0]
13	load2	1	Loads i (value: 2) onto the stack.	[0, 2]
14	load2	1	Loads i (value: 2) onto the stack again.	[0, 2, 2]
15	mul	1	Pops two values and multiplies them (2 * 2 = 4).	[0, 4]
16	add	1	Pops two values and adds them (0 + 4 = 4).	[4]
17	store3	1	Pops 4 and stores it into s (slot 3).	[]
18	inc 2 1	3	Increments local variable i (slot 2) by 1 (2 -> 3).	[]
21	jmp -14	3	Jumps back to the loop condition check at address 7.	[]

Figure 4: 4b

7	load2	1	Loads i (value: 3) onto the stack.	[3]
8	load1	1	Loads max (value: 4) onto the stack.	[3, 4]
9	jgt 15	3	Compares if 3 > 4. False, execution continues to loop body.	[]
12	load3	1	Loads s (value: 4) onto the stack.	[4]
13	load2	1	Loads i (value: 3) onto the stack.	[4, 3]
14	load2	1	Loads i (value: 3) onto the stack.	[4, 3, 3]
15	mul	1	Multiplies top two elements ($3 * 3 = 9$).	[4, 9]
16	add	1	Adds top two elements ($4 + 9 = 13$).	[13]
17	store3	1	Stores 13 back into s (slot 3).	[]
18	inc 2 1	3	Increments local variable i by 1 ($3 \rightarrow 4$).	[]
21	jmp -14	3	Jumps back to the loop condition check at address 7.	[]

Figure 5: 4c

7	load2	1	Loads i (value: 4) onto the stack.	[4]
8	load1	1	Loads max (value: 4) onto the stack.	[4, 4]
9	jgt 15	3	Compares if 4 > 4. False, execution continues to loop body.	[]
12	load3	1	Loads s (value: 13) onto the stack.	[13]
13	load2	1	Loads i (value: 4) onto the stack.	[13, 4]
14	load2	1	Loads i (value: 4) onto the stack.	[13, 4, 4]
15	mul	1	Multiplies top two elements ($4 * 4 = 16$).	[13, 16]
16	add	1	Adds top two elements ($13 + 16 = 29$).	[29]
17	store3	1	Stores 29 back into s (slot 3).	[]
18	inc 2 1	3	Increments local variable i by 1 ($4 \rightarrow 5$).	[]
21	jmp -14	3	Jumps back to the loop condition check at address 7.	[]

Figure 6: 4d

7	load2	1	Loads i (value: 5) onto the stack.	[5]
8	load1	1	Loads max (value: 4) onto the stack.	[5, 4]
9	jgt 15	3	Compares if 5 > 4. True! Jumps over loop body to address $9 + 15 = 24$.	[]
24	load3	1	Loads the final sum s (value: 29) onto the stack.	[29]
25	exit	1	Cleans up sum2 frame, restores frame pointer of main.	[29]
26	return	1	Pops return address 37 from MStack and jumps there.	[29]
37	const0	1	Pushes formatting width 0 onto the stack.	[29, 0]
38	print	1	Consumes 0 and 29, printing 29 to output console.	[]
39	exit	1	Cleans up main's stack frame.	[]
40	return	1	Exits the main program execution safely.	

Figure 7: 4e

Stack Trace Analysis during the Execution of `sum2`

Using the provided interpreter debug trace, the dynamic behavior of the method stack (MStack) and expression stack (EStack) during execution can be traced step-by-step:

A. Argument Preparation and Method Invocation in `main`

```
29: enter   |  
32: const2  | 2  
33: const4  | 2 4  
34: call    | 2 4
```

- **Instruction 29** (`enter 0 0`): Prepares an empty stack frame for `main` (since it has 0 parameters and 0 local variables).
- **Instructions 32 & 33** (`const2`, `const4`): The actual parameters for the upcoming method call (2 and 4) are pushed sequentially onto the EStack. The state of the expression stack becomes: [2, 4].
- **Instruction 34** (`call -34`): The return address 0 is pushed onto the MStack to ensure the VM knows where to resume after returning.

B. Entering the Callee Method (`sum2`)

```
0: enter   |
```

- **Instruction 0** (`enter 2 4`): The old frame pointer of `main` is backed up onto the MStack, and a new frame with 4 local variable slots is instantiated. The argument values 4 and 2 are popped from the EStack and written into local variable slots 1 (`max`) and 0 (`min`). Consequently, the EStack is temporarily completely empty.
- *Execution Note*: The subsequent loop iterates three times to compute the sum of squares: $s = 0 + 2^2 + 3^2 + 4^2 = 4 + 9 + 16 = 29$.

C. Method Termination and Return Preparation

Once the loop condition terminates, the final computed result is prepared for return:

```
24: load3   | 29  
25: exit    | 29  
26: return  | 29
```

- **Instruction 24** (`load3`): Loads the computed value of the variable `s` from local slot 3 onto the EStack. The expression stack state becomes: [29].
- **Instruction 25** (`exit`): The stack frame belonging to `sum2` is removed from the MStack. The frame pointer is successfully restored to point back to the stack frame of `main`. Crucially, the return value 29 remains safely preserved on top of the EStack.
- **Instruction 26** (`return`): Pops the target address 37 from the MStack and shifts control flow back into `main`.

D. Processing the Returned Value in `main`

```
37: const0  | 29 0  
38: print   | 29
```

- **Instruction 37** (`const0`): Pushes the formatting parameter (output width 0) onto the EStack. The stack state becomes: [29, 0].
- **Instruction 38** (`print`): Consumes both the output width parameter (0) and the value (29) from the EStack, printing the final answer 29 to the standard console output. The expression stack is left empty.

Output from MJ.Compiler MethodCall.mj

```

0: enter 2 4
3: const0
4: store3
5: load0
6: store2
7: load2
8: load1
9: jgt 15
12: load3
13: load2
14: load2
15: mul
16: add
17: store3
18: inc 2 1
21: jmp -14
24: load3
25: exit
26: return
27: trap 1
29: enter 0 0
32: const2
33: const4
34: call -34
37: const0
38: print
39: exit
40: return

```

Output from MJ.Run with -debug option

```

29: enter      |
32: const2    | 2
33: const4    | 2 4
34: call      | 2 4
0: enter      |
3: const0     | 0
4: store3     |
5: load0      | 2
6: store2     |
7: load2      | 2
8: load1      | 2 4
9: jgt        |
12: load3     | 0
13: load2     | 0 2
14: load2     | 0 2 2
15: mul       | 0 4
16: add       | 4
17: store3    |
18: inc       |
21: jmp       |
7: load2     | 3
8: load1     | 3 4
9: jgt        |
12: load3     | 4
13: load2     | 4 3
14: load2     | 4 3 3
15: mul       | 4 9
16: add       | 13
17: store3    |
18: inc       |
21: jmp       |
7: load2     | 4
8: load1     | 4 4
9: jgt        |
12: load3     | 13
13: load2     | 13 4
14: load2     | 13 4 4
15: mul       | 13 16
16: add       | 29
17: store3    |
18: inc       |
21: jmp       |
7: load2     | 5
8: load1     | 5 4
9: jgt        |
24: load3     | 29
25: exit      | 29
26: return    | 29
37: const0    | 29 0
38: print     | 29
39: exit      |
40: return    |

```

MicroJava – Grammatik

```

Program      = "program" ident { ConstDecl | VarDecl | ClassDecl }
              "{" { MethodDecl } "}".

ConstDecl   = "final" Type ident "=" ( number | charConst ) ";".
VarDecl     = Type ident { "," ident } ";".
ClassDecl   = "class" ident "{" { VarDecl } ";".
MethodDecl  = ( Type | "void" ) ident "(" [ FormPars ] ")"
              { VarDecl } Block.
FormPars    = Type ident { "," Type ident }.
Type        = ident [ "[" "]" ].

Block       = "{" { Statement } ";".
Statement   = Designator ( Assignop Expr | ActPars | "++" | "--" ) ";"
              | "if" "(" Condition ")" Statement [ "else" Statement ]
              | "while" "(" Condition ")" Statement
              | "switch" "(" Expr ")" "{" { "case" Expr ":" { Statement } }
              [ "default" ":" { Statement } ] "}"
              | "break" ";"
              | "return" [ Expr ] ";"
              | "read" "(" Designator ")" ";"
              | "print" "(" Expr [ "," number ] ")" ";"
              | Block
              | ";"

Assignop    = "=" | "+=" | "-=" | "*=" | "/=" | "%=".
ActPars     = "(" [ Expr { "," Expr } ] ")".

Condition   = CondTerm { "||" CondTerm }.
CondTerm    = CondFact { "&&" CondFact }.
CondFact    = Expr Relop Expr.
Relop       = "==" | "!=" | ">" | ">=" | "<" | "<="

Expr        = [ "-" ] Term { Addop Term }.
Term        = Factor { Mulop Factor }.
Factor      = Designator [ ActPars ]
              | number
              | charConst
              | "new" ident [ "[" Expr "]" ]
              | "(" Expr ")".
Designator  = ident { "." ident | "[" Expr "]" }.
Addop       = "+" | "-".
Mulop       = "*" | "/" | "%".

```